MLE ASSIGNMENT - 3

ASSIGNMENT

On

CALIFORNIA HOUSING PRICE PREDICTION

Submitted to

MIT ADT University, Pune

in partial fulfillment of the requirements for the Assignment of the course

Machine Learning Essentials

Submitted by

PRANAV WAGH ADT23SOCB0752

Roll No : 23

Under the Guidance of

Prof. Vijaya Patil

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MIT SCHOOL OF COMPUTING, MIT ADT UNIVERSITY

RAJBAUG, LONI KALBHOR, PUNE – 412201

THIRD YEAR - SOC **A.Y 2025-26** **SEMESTER-II**

MLE ASSIGNMENT - 3

1. **A data scientist working for a real-estate company needs to predict house prices using demographic and geographical factors from the California Housing dataset.**
   The model must:
   handle missing values, scale features, split into train/test sets, train a regression model, evaluate performance, generate predictions. The goal is to automate the pipeline so it can be reused in future projects.

Question 1 —
Load Dataset Load the California Housing dataset and print:
• Feature names
• Target name
• Number of samples and features

```python
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import joblib  # For saving model (optional)
```

```python
# Load from attached file (matches scikit-learn California Housing)
df = pd.read_csv('D:/MLE/Assignment_3/housing.csv')

# Feature and target info (standard for this dataset)
feature_names = ['longitude', 'latitude', 'housing_median_age', 'total_rooms',
                 'total_bedrooms', 'population', 'households', 'median_income']
target_name = 'median_house_value'

X = df[feature_names]
y = df[target_name]

print("Feature names:", feature_names)
print("Target name:", target_name)
print("Number of samples:", len(df))
print("Number of features:", len(feature_names))
print("Dataset shape:", df.shape)
```

```
Feature names: ['longitude', 'latitude', 'housing_median_age', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_income']
Target name: median_house_value
Number of samples: 20640
Number of features: 8
Dataset shape: (20640, 10)
```

Question 2 — Convert to DataFrame Convert data into a Pandas DataFrame and display first 10 rows.

MLE ASSIGNMENT - 3

```
print("First 10 rows:")
print(df.head(10))
```

```
First 10 rows:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                41.0        880.0           129.0
1    -122.22     37.86                21.0       7099.0          1106.0
2    -122.24     37.85                52.0       1467.0           190.0
3    -122.25     37.85                52.0       1274.0           235.0
4    -122.25     37.85                52.0       1627.0           280.0
5    -122.25     37.85                52.0        919.0           213.0
6    -122.25     37.84                52.0       2535.0           489.0
7    -122.25     37.84                52.0       3104.0           687.0
8    -122.26     37.84                42.0       2555.0           665.0
9    -122.25     37.84                52.0       3549.0           707.0

   population  households  median_income  median_house_value ocean_proximity
0       322.0       126.0         8.3252            452600.0        NEAR BAY
1      2401.0      1138.0         8.3014            358500.0        NEAR BAY
2       496.0       177.0         7.2574            352100.0        NEAR BAY
3       558.0       219.0         5.6431            341300.0        NEAR BAY
4       565.0       259.0         3.8462            342200.0        NEAR BAY
5       413.0       193.0         4.0368            269700.0        NEAR BAY
6      1094.0       514.0         3.6591            299200.0        NEAR BAY
7      1157.0       647.0         3.1200            241400.0        NEAR BAY
8      1206.0       595.0         2.0804            226700.0        NEAR BAY
9      1551.0       714.0         3.6912            261100.0        NEAR BAY
```

Question 3 — Check Missing Values Check for missing values and report findings.

```
print("Missing values per column:")
print(df.isnull().sum())
```

[4]

```
Missing values per column:
longitude               0
latitude                0
housing_median_age      0
total_rooms             0
total_bedrooms        207
population              0
households              0
median_income           0
median_house_value      0
ocean_proximity         0
dtype: int64
```

MLE ASSIGNMENT - 3

Question 4 — Handle Missing Values

Impute missing values using: SimpleImputer(strategy="median")

```
imputer = SimpleImputer(strategy='median')
X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=feature_names)
print("Shape after imputation:", X_imputed.shape)
print("Missing values after:", X_imputed.isnull().sum().sum())
```

[5]

```
Shape after imputation: (20640, 8)
Missing values after: 0
```

Question 5 — Train-Test Split

Split dataset into: 80% training 20% testing

```
X_train, X_test, y_train, y_test = train_test_split(
    X_imputed, y, test_size=0.2, random_state=42
)
print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
```

[6]

```
Training samples: 16512
Test samples: 4128
```

Question 6 — Feature Scaling

Apply StandardScaler and explain why scaling is needed.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training data mean after scaling (should ~0):", np.mean(X_train_scaled, axis=0).round(2))
print("Training data std after scaling (should ~1):", np.std(X_train_scaled, axis=0).round(2))
```

[7]

```
Training data mean after scaling (should ~0): [ 0.  0. -0.  0. -0. -0. -0. -0.]
Training data std after scaling (should ~1): [1. 1. 1. 1. 1. 1. 1. 1.]
```

Question 7 — Train Model

Train **Linear Regression** on the processed dataset.

```
model = LinearRegression()
model.fit(X_train_scaled, y_train)
print("Model coefficients shape:", model.coef_.shape)
print("Model trained successfully!")
```

[8]

```
Model coefficients shape: (8,)
Model trained successfully!
```

Question 8 — Model Evaluation

Evaluate model using:

• Mean Squared Error (MSE)

• Root Mean Squared Error (RMSE)

• R^2

Score Interpret the results in 3–4 lines.

```
y_pred = model.predict(X_test_scaled)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R² Score: {r2:.4f}")

print("\nInterpretation:")
print("- RMSE 71133 means average error of ~$71k (target in $ units).")
print("- R² 0.6139: Model explains 61% of house price variance; solid linear baseline.")
print("- MSE 5.06e9 shows scale of squared errors; typical for $200k+ prices.")
print("- Improve with feature engineering (rooms/household) or RandomForestRegressor.")
```

[10]

```
MSE: 5059928371.17
RMSE: 71133.17
R² Score: 0.6139

Interpretation:
- RMSE 71133 means average error of ~$71k (target in $ units).
- R² 0.6139: Model explains 61% of house price variance; solid linear baseline.
- MSE 5.06e9 shows scale of squared errors; typical for $200k+ prices.
- Improve with feature engineering (rooms/household) or RandomForestRegressor.
```

With an **RMSE of 71,133** indicating an average error of approximately **$71k** and an **MSE of 5.06 * 10^9** reflecting the scale of squared errors for typical home prices, the model achieves an **R^2 of 0.6139**, meaning it explains **61%** of house price variance as a solid linear baseline that could be further improved through **feature engineering** or a **RandomForestRegressor**.

Question 9 — Save Predictions

Generate predictions on the test set and save them into: predictions.csv

```
predictions_df = pd.DataFrame({
    'actual': y_test.values,
    'predicted': y_pred
})
predictions_df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv (first 5 rows):")
print(predictions_df.head())
```

[11]

```
Predictions saved to predictions.csv (first 5 rows):
     actual      predicted
0   47700.0   63736.591338
1   45800.0  154344.594319
2  500001.0  253073.194287
3  218600.0  263507.746536
4  278000.0  266883.359611
```

Question 10 — Reflection

Answer briefly:

• Which pipeline step was most useful?

Most useful pipeline step: Feature scaling with StandardScaler was most critical. Linear Regression is highly sensitive to feature scales—total_rooms (thousands) would dominate housing_median_age (0-52) without it, leading to poor coefficients and predictions.

• Where could errors occur?

- Data leakage: Scaling before train/test split (fixed by splitting first).
- Imputation on categoricals: ocean_proximity ignored here; needs encoding if used.
- Target leakage: Including households/population ratios that proxy target.
- Random state inconsistency across runs affecting reproducibility.

• How could accuracy be improved?

- Add feature engineering: rooms-per-household, bedrooms-per-room ratios.
- Polynomial features or tree models (RandomForestRegressor, XGBoost).
- Encode ocean_proximity as dummy variables (+5% R^2 expected).
- Hyperparameter tuning via GridSearchCV or cross-validation.

CONCLUSION:-

This assignment successfully implemented a complete end-to-end ML pipeline on the California Housing dataset using scikit-learn. Key accomplishments include median imputation for missing total_bedrooms (~207 values), StandardScaler normalization, 80/20 train-test split, Linear Regression training, and evaluation yielding RMSE=71,133 and R^2=0.6139 (61% variance explained). The pipeline generated predictions.csv (4,128 test predictions) ready for production deployment. Critical learning: feature scaling was essential to prevent feature dominance, establishing a solid baseline for advanced regression models like Random Forest or XGBoost.